Write-up: BookLeaf Author Support Portal

Naman Panicker · Live demo: naman-panicker-book-leaf.vercel.app

What I prioritised

An Author wants a specific answer about their own Book, and an Admin wants to give one quickly and correctly. I built the core Ticket flow first, then the AI, then the Admin workflow around it.

I treated AI as an assistant to a person, not a replacement. The AI classifies every Ticket and writes a Draft, and an Admin checks it before anything reaches an Author. A wrong royalty figure does real damage, so anything that can be computed is computed in code: Next Payout Date, Below Threshold, Overdue. The model only puts those facts into BookLeaf's voice. Policy works the same way: ISBN errors and royalty Tickets from Overdue Authors are raised to at least High by a rule, whatever the model says.

Tickets are saved before any AI call, so an outage never stops an Author. Admins see what failed, can classify by hand or retry, and can always write the reply themselves.

Trade-offs

- **One small model.** Gemini Flash-Lite handles both jobs, because they're constrained and grounded in supplied facts. Its free tier keeps the demo free, but real Author data would need the paid tier.
- **Sending less to the model.** Each Draft carries only its Category's section, 13–26% of the Knowledge Base, plus the last six Messages. Drafts are cached and written only when an Admin opens a Ticket. The cost is that a misclassified Ticket gets the wrong section, which is another reason an Admin checks.
- **Polling over streaming.** An SSE prototype dropped at irregular intervals through the Vercel rewrite. Polling every five seconds is simple and recovers by itself, at the cost of a few seconds' delay and steady requests.
- **Separate Next.js and Express deployments.** More to run than one app, but background classification needs a long-running process.
- **Self-assignment without a handover.** I designed a request-and-accept Takeover but cut it for time. For now, a Ticket held by an absent Admin stays stuck with them.
- **Testing.** API tests run against a real database, and business rules have unit tests. There are no browser tests or AI evaluation suite yet.

Taking it to production

1. **Measure the AI first.** Stored suggestions and overrides already show how often Admins disagree with it. I'd add a labelled set of real Tickets and track how much Admins edit Drafts.
2. **Make it sturdier.** A durable job queue for AI work, rate limiting, password reset, monitoring, the paid AI tier, and hosting with no cold starts.
3. **Finish the workflow.** Takeover, real attachments, and email notifications, because Authors won't keep a tab open.
4. **Then let the AI do more, gradually.** Once the numbers justify it, routine questions with clear answers from the Author's own data, such as "when is my next Payout?", could be answered without an Admin. Money disputes and BookLeaf's mistakes would still go to a person, and ISBN errors would route straight to the production team. That's the real gain in Admin time, but I'd earn it with data.